05-deployment

05 — Deployment

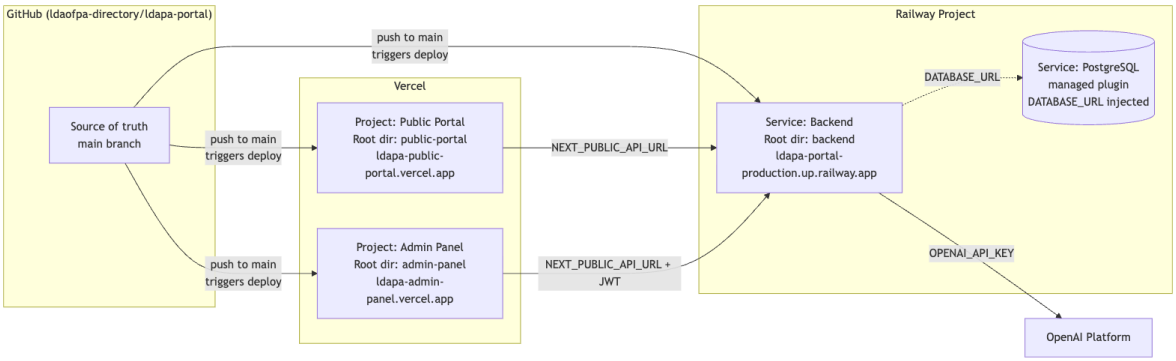
Target audience: engineers or semi-technical owners who need to understand how the live system was deployed and how to manage it going forward.

Live URLs

Component	URL
Public Portal	https://ldapa-public-portal.vercel.app
Admin Panel	https://ldapa-admin-panel.vercel.app
Backend API	https://ldapa-portal-production.up.railway.app
Backend Swagger UI	https://ldapa-portal-production.up.railway.app/docs
GitHub repo	https://github.com/ldaofpa-directory/ldapa-portal
Railway project	https://railway.com/project/dd7377a4-0528-4aa5-bc96-cf5f08f015b9

---

1. Deployment Topology



Deployment Diagram

Three services across two platforms, all driven by one GitHub repository:

- **GitHub** holds the source of truth. Pushes to `main` trigger deploys in both Railway and Vercel.
- **Railway** runs the FastAPI backend and the managed PostgreSQL instance that it connects to.

- **Vercel** runs the two Next.js apps (public portal + admin panel), each as its own Vercel project pointing at a different subdirectory of the same repo.

There are **no custom domains** at the time of writing. All traffic flows through the default `*.vercel.app` and `*.up.railway.app` URLs.

---

2. GitHub Repository

- Repo URL: https://github.com/ldaofpa-directory/ldapa-portal
- Default branch: `main` — deploys are wired to this branch on every platform.
- This is a **fork** of the original development repo. Ownership was transferred to the `ldaofpa-directory` GitHub organization for handoff.

Access: log in to GitHub with the `directory@ldaofpa.org` Google account. The account owns the organization and the repo. Any future developer will need to be invited as a collaborator.

Protected branch recommendation (optional): enable branch protection on `main` to require pull requests rather than direct pushes. Settings → Branches → Add branch protection rule → Branch name pattern: `main` → check "Require a pull request before merging".

---

3. Railway — Backend + PostgreSQL

3.1 Project layout

Inside the Railway project (https://railway.com/project/dd7377a4-0528-4aa5-bc96-cf5f08f015b9) there are two services:

Service	Type	Notes
Backend	GitHub-deployed Python app	Root directory: `backend/`
Postgres	Managed database plugin	Created via **+ New** → **Database** → **PostgreSQL**

Railway auto-injects `DATABASE_URL` from the Postgres service into the backend service via Railway's "reference variables" system. No wiring needed.

3.2 How the backend was deployed

The one-time setup, as it was performed:

1. On Railway → **New Project** → **Deploy from GitHub Repo** → chose `ldaofpa-directory/ldapa-portal`.
2. Service settings → **Root Directory**: `backend`.
3. Added a PostgreSQL plugin to the same project (**+ New** → **Database** → **PostgreSQL**).

4. Set environment variables on the backend service (see §3.4).
5. Generated a public domain: **Settings** → **Networking** → **Generate Domain** → `ldapa-portal-production.up.railway.app`.
6. First deploy completed automatically; `init_db()` created all tables and auto-imported the CSV from the repo.

Ongoing deploys are triggered automatically every time `main` is pushed.

3.3 Build and run

The build uses Railway's **Nixpacks** buildpack (no Dockerfile). Python 3.11 is specified by `backend/runtime.txt`.

Start command — configured in `railway.json`:

```
{
  "build":  { "builder": "NIXPACKS" },
  "deploy": {
    "startCommand": "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}",
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 10
  }
}
```

`backend/Procfile` is a secondary/backup that Railway also recognizes:

```
web: uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}
```

`backend/requirements.txt` pins all Python dependencies — Railway runs `pip install -r requirements.txt` automatically.

3.4 Environment variables (backend service)

Set in Railway **Variables** tab:

Variable	Value	How to obtain
`DATABASE_URL`	`postgresql://postgres:...@...railway.internal:5432/railway`	Auto-injected by the Postgres plugin — do not set manually
`OPENAI_API_KEY`	`sk-proj-...`	Created in the

Variable	Value	How to obtain
		OpenAI Platform dashboard — see Doc 6
`JWT_SECRET`	64-character random hex	Run `openssl rand -hex 32` locally and paste
`CORS_ORIGINS`	`https://ldapa-public-portal.vercel.app,https://ldapa-admin-panel.vercel.app`	Comma-separated list of frontends allowed to call the API
`LLM_MODEL`	`gpt-5-mini`	(optional — this is the default)
`PORT`	*(auto-injected)*	Railway sets this at runtime

Railway auto-redeploys when any variable changes.

3.5 Health check and verification

After every deploy, visit `https://ldapa-portal-production.up.railway.app/api/health` — expect `{"status":"ok"}`. Also confirm the admin login works end-to-end (see §7).

3.6 Logs

Railway → service → **Deployments** tab → click a deployment → **View Logs**. Both stdout and stderr are captured. Failed requests will show Python tracebacks here.

3.7 PostgreSQL access

To run SQL directly (for analytics, backups, or one-off fixes):

- **From the Railway dashboard:** Postgres service → **Data** tab → query editor. Safest option — no local tools needed.

- **From your terminal:** Postgres service → **Variables** → copy `DATABASE_URL` → `psql "<url>"`.
- **From the seed script:** `cd backend && DATABASE_URL="<url>" python seed_production.py` (the script refuses to run if the table already has rows).

3.8 Manual database backups (free tier)

On the free Railway plan, automatic snapshots are not included. Take a manual `pg_dump` monthly (or before any risky migration):

```
# Copy DATABASE_URL from Railway dashboard → Postgres service → Variables
pg_dump "$DATABASE_URL" > ldapa-backup-$(date +%Y%m%d).sql
```

Store the resulting `.sql` file somewhere safe (Google Drive under the `directory@ldaofpa.org` account is a reasonable choice — backups are small, under 10 MB).

To restore:

```
psql "$DATABASE_URL" < ldapa-backup-YYYYMMDD.sql
```

3.9 Scaling up

Railway's free / hobby plan is sufficient for the current traffic. Signs you'd want to upgrade:

- Backend logs show "memory limit exceeded" restarts.
- Postgres shows slow queries or "too many connections" errors.
- You need guaranteed uptime SLA.

The paid **Pro** plan (currently $20/month) bumps resources and enables automatic daily backups. To upgrade: Railway → project → **Settings** → **Plan**.

---

4. Vercel — Public Portal

4.1 Project configuration

On Vercel (logged in as `directory@ldaofpa.org`), the public portal is a **separate Vercel project** pointing at the same GitHub repo, with these settings:

Setting	Value
Project name	`ldapa-public-portal`
Production URL	https://ldapa-public-portal.vercel.app
Framework preset	Next.js (auto-detected)
Root directory	`public-portal`

Setting	Value
Production branch	`main`
Node.js version	20.x (Vercel default)

4.2 How it was deployed

1. Vercel → **Add New** → **Project** → import `ldaofpa-directory/ldapa-portal`.
2. Framework: Next.js (auto).
3. **Root Directory**: `public-portal`.
4. Set environment variable `NEXT_PUBLIC_API_URL = https://ldapa-portal-production.up.railway.app`.
5. Click **Deploy** — Vercel builds and publishes.

4.3 Environment variables

Variable	Value
`NEXT_PUBLIC_API_URL`	`https://ldapa-portal-production.up.railway.app`

Vercel rebuilds the project on every push to `main`.

4.4 Verifying

Visit https://ldapa-public-portal.vercel.app, click into `/chat`, send a test message. You should see provider cards render. If not, check the browser console for CORS or 401 errors and verify the Railway backend is up.

---

5. Vercel — Admin Panel

5.1 Project configuration

A second Vercel project, sibling to the public portal:

Setting	Value
Project name	`ldapa-admin-panel`
Production URL	https://ldapa-admin-panel.vercel.app
Framework preset	Next.js
Root directory	`admin-panel`
Production branch	`main`

5.2 Environment variables

Variable	Value
`NEXT_PUBLIC_API_URL`	`https://ldapa-portal-production.up.railway.app`

5.3 Verifying

Visit https://ldapa-admin-panel.vercel.app, log in with `directory@ldaofpa.org` and the handed-over password, confirm the dashboard loads with provider counts and chat-volume chart.

---

6. Wire-Up Between Platforms

6.1 CORS

The backend accepts cross-origin calls only from URLs listed in the `CORS_ORIGINS` env var. Current value:

```
https://ldapa-public-portal.vercel.app,https://ldapa-admin-panel.vercel.app
```

If a Vercel URL ever changes (e.g. after renaming a project), update `CORS_ORIGINS` on Railway — it will redeploy automatically.

6.2 NEXT_PUBLIC_API_URL

Both Vercel projects point at the Railway backend via `NEXT_PUBLIC_API_URL`. If Railway ever issues a new domain (or you add a custom domain), update this variable on **both** Vercel projects and trigger a redeploy.

6.3 Typical change → deploy cycle

1. A developer pushes to `main` on GitHub.
2. Railway rebuilds the backend (~1–2 min). Vercel rebuilds each frontend in parallel (~1 min each).
3. Changes are live.

For PR preview deploys on Vercel: any branch opened as a PR gets its own preview URL automatically (e.g., `ldapa-public-portal-git-feature-x.vercel.app`). Railway does not offer preview environments on the free tier.

---

7. Post-Deploy Verification Checklist

After any significant deploy, confirm:

- ☐ `GET /api/health` returns `{"status":"ok"}`.

- [ ] Railway **Deployments** tab shows the latest commit as “Deployed”.
- [ ] Public portal loads; sending a chat message returns an assistant response + provider cards.
- [ ] Admin login at https://ldapa-admin-panel.vercel.app works with `directory@ldaofpa.org` / current password.
- [ ] `/overview` on the admin panel shows a provider count > 0.
- [ ] No red errors in Railway logs for the last 5 minutes.

---

8. Ongoing Management Tasks

8.1 Rotating the admin password

1. Log in to the admin panel.
2. (When the settings page exposes it) change the password.
3. Otherwise, run SQL in Railway’s Postgres console:

   ```
   UPDATE admin_users
   SET password_hash = '$2b$12$...new-bcrypt-hash-here...'
   WHERE email = 'directory@ldaofpa.org';
   ```

   Generate the bcrypt hash locally with Python:

   ```
   import bcrypt
   bcrypt.hashpw(b"new-password", bcrypt.gensalt()).decode()
   ```

   Important: `init_db()` re-asserts the canonical hash on every boot (see `backend/app/database.py`). So if you rotate in Postgres only, the next redeploy will reset it. Either update `admin_hash` in `database.py` to match, or accept that the rotation lasts only until the next deploy.

8.2 Rotating the OpenAI API key

See **Doc 6 — Handoff, §OpenAI** for the end-to-end steps. In short: create a new key in the OpenAI dashboard, paste into Railway `OPENAI_API_KEY`, revoke the old key.

8.3 Adding a new admin user

Insert directly into `admin_users` (no self-service UI yet):

```
INSERT INTO admin_users (id, email, password_hash, name)
VALUES ('admin2', 'someone@ldaofpa.org', '<bcrypt hash>', 'Someone');
```

8.4 Uploading a refreshed CSV from Brilliant Directories

1. Export a fresh members CSV from Brilliant Directories.
2. Log in to the admin panel → **Import / Export**.
3. Upload the file. The preview page will show valid rows, warnings, and errors.
4. Review carefully — the importer does **not** deduplicate. If you're replacing rather than adding, you may need to archive or delete stale providers first.
5. Click **Confirm** to insert.

8.5 Updating the LLM model

If OpenAI releases a better/cheaper model:

1. Update the `LLM_MODEL` env var on Railway (e.g., `gpt-5-mini` → `gpt-5-nano`).
2. Railway redeploys automatically.
3. Test a chat conversation end-to-end before assuming the change is safe — different models vary in prompt adherence.

8.6 Changing CORS after a URL change

If you rename a Vercel project or add a custom domain:

1. Update `CORS_ORIGINS` on Railway.
2. Update `NEXT_PUBLIC_API_URL` on Vercel (if the backend URL changed).
3. Redeploy the affected Vercel project.

---

9. Rolling Back a Bad Deploy

9.1 Backend (Railway)

Railway keeps every past deployment. To roll back:

1. Railway → backend service → **Deployments** tab.
2. Find a known-good deployment.
3. Click the menu → **Redeploy**.

9.2 Frontend (Vercel)

Vercel likewise keeps every deployment:

1. Vercel → project → **Deployments** tab.
2. Click the menu on a prior deployment → **Promote to Production**.

9.3 Database (Postgres)

If a bad release corrupted data, restore from the most recent `pg_dump` — see §3.8.

---

10. Adding a Custom Domain (When Ready)

Neither platform currently has a custom domain. If/when LDA of PA wants, e.g., `portal.ldaofpa.org` and `api.ldaofpa.org`:

10.1 Vercel

1. Vercel → project → **Settings** → **Domains** → **Add**.
2. Enter `portal.ldaofpa.org` (or similar) — Vercel shows the DNS records to add.
3. Update the DNS at the LDA of PA domain registrar with the given CNAME/A records.
4. Wait for Vercel to verify and provision SSL (usually <5 min).
5. Repeat for the admin subdomain.

10.2 Railway

1. Railway → backend service → **Settings** → **Networking** → **Custom Domain**.
2. Enter `api.ldaofpa.org`.
3. Add the CNAME at the DNS registrar.
4. Wait for Railway to provision SSL.

10.3 After adding domains

- Update `NEXT_PUBLIC_API_URL` on both Vercel projects to the new backend domain.
- Update `CORS_ORIGINS` on Railway to include the new frontend domains.
- Both sides redeploy automatically.

---

11. Cost Profile

Current monthly cost at today's traffic:

Service	Plan	Monthly
Railway	Free (Trial / Starter credit)	$0 until credit exhausted
Vercel (public portal)	Hobby (free)	$0
Vercel (admin panel)	Hobby (free)	$0
OpenAI	Pay-as-you-go	~$5–15 at current chat volume
Total		**~$5–15**

Warning: the Railway free trial credit is time-limited and will expire. When it does, the backend and database will stop running. See **Doc 6 — Handoff** for the step-by-step on adding payment info to Railway and OpenAI before this happens.

---

12. Related Documents

- **01 — Technical Requirements and Design**
- **02 — System Architecture**
- **03 — APIs**
- **04 — Data Modeling & DB Population**
- **06 — Handoff**